

SMARTPARK KE — ALGORITHMIC SPECIFICATION

Complete Pseudocode Breakdown for All 11 Polyglot Modules

Module 1 — Vehicle Entry & Registration (Python + C++)

Data Structure: Hash Map [O(1) lookup]

```
Algorithm EntryRegistration(plate, type):
    clean_plate = SanitizeAndUppercase(plate)
    if ActiveSessionMap.contains(clean_plate):
        throw DuplicateException("Vehicle already inside")
    slot = SlotEngineClient.requestNearestSlot()
    session = DB.insertParkingSession(clean_plate, slot.id, now())
    ActiveSessionMap.put(clean_plate, session)
    TimeTrackerHeap.insert(session.entry_time, session)
    WebSocket.emit("slot_state_changed", {slot.id, "OCCUPIED", clean_plate})
    return session
```

Module 2 — Nearest Slot Allocation Engine (C++)

Data Structures: Min-Heap [O(log n)], Graph Dijkstra [O((V+E)log V)], Doubly Linked List [O(1)]

```
Algorithm ExtractMinSlot():
    if freeMinHeap.empty(): return NULL
    nearest = freeMinHeap.extractMin() // O(log n)
    rowList = zoneLists[nearest.zoneId]
    rowList.updateStatus(nearest.slotId, "OCCUPIED") // O(1)
    return nearest
```

```
Algorithm ReleaseSlot(slotId):
    slot = allSlots[slotId]
    freeMinHeap.insert(slot) // O(log n)
    zoneLists[slot.zoneId].updateStatus(slotId, "FREE") // O(1)
```

Module 3 — Real-Time Visual Slot Display (Python + JS)

Data Structure: In-Memory 2D Matrix Array [O(1) lookup]

```
Algorithm PushSlotDiff(slotId, status, plate):
    matrix[row][col].status = status
    matrix[row][col].plate = plate
    WebSocket.emit("slot_state_changed", {slotId, status, plate})
```

Module 4 — Continuous Time Tracking (Python + C++)

Data Structure: Hash Map (plate -> entry) + Min-Heap ordered by entry_time [O(1) peek]

```
Algorithm DetectOverstay(threshold_minutes):
    alerts = []
    while TimeHeap.notEmpty() and (now() - TimeHeap.peek().entryTime) >= threshold_minutes:
        rec = TimeHeap.pop()
        if rec.active:
            alerts.append(rec)
    return alerts
```

Module 5 — Tariff Engine & Fee Calculation (Java)

Data Structure: Small Dynamic Tariff Lookup Table [0(1) scan]

```
Algorithm CalculateFee(minutes):
    m = max(0, minutes)
    for tier in tariffTable:
        if m <= tier.maxMinutes: return tier.fee
    return 500.00
```

Module 6 — Safaricom Daraja M-Pesa STK Push (Java / Spring Boot)

Data Structure: FIFO Queue (ConcurrentLinkedQueue) for Webhook Payloads

```
Algorithm DispatchSTK(sessionId, phone, fee):
    pwd = Base64(Shortcode + Passkey + Timestamp("yyyyMMddHHmmss"))
    reqId = DarajaAPI.sendSTKPush(phone, fee, pwd)
    pendingMap.put(reqId, {sessionId, fee, phone})
    return reqId
```

```
Algorithm OnWebhook(payload):
    fifoQueue.offer(payload) // FIFO Queue
    while not fifoQueue.isEmpty():
        event = fifoQueue.poll()
        if event.resultCode == 0:
            MarkPaid(event.checkoutRequestId)
            TriggerBarrierActuator(event.sessionId)
            NotificationQueue.enqueue(event)
```

Module 7 — Barrier Actuator Finite State Machine (C++)

Data Structure: LIFO Stack (std::stack) for Safety Auditing

```
Algorithm HandlePaymentConfirmed(sessionId):
    if state == CLOSED or state == CLOSING:
        state = OPENING
        auditStack.push({transitionId++, from: "CLOSED", to: "OPENING", trigger: "PAYMENT"})
        ActuatorHardware.open()
        state = OPEN
        return true
    return false
```

Module 8 — Relational Database Management (Python / SQLAlchemy)

Data Structure: B-Tree Index on Timestamp & Foreign Keys; Repository Pattern

```
Algorithm RepoWrite(modelInstance):
    db = SessionLocal()
    try: db.add(modelInstance); db.commit()
    catch e: db.rollback(); raise e
    finally: db.close()
```

Module 9 — Admin Analytics & Reporting (Python)

Data Structure: B-Tree Range Queries on Timestamps

```
Algorithm QueryRevenue(start, end):
    return DB.query(sum(fee_charged))
        .filter(entry_time >= start, exit_time <= end, paid == True)
```

Module 10 — Authentication & Role-Based Access Control (Python)

Data Structure: Hash Map Active Sessions; Bcrypt Hash

```
Algorithm Login(username, pass):
    user = DB.find(username)
    if user and bcrypt_verify(pass, user.hash):
        session_token = uuid()
        activeSessionMap[session_token] = user.role
    return session_token
```

Module 11 — Notification Dispatcher (Java / Spring Boot + Python)

Data Structure: FIFO Queue (ConcurrentLinkedQueue) for SMS Receipts

```
Algorithm EnqueueNotification(phone, receipt, amount):
    job = {phone, receipt, amount, text: "SmartPark KE: Confirmed Kshs " + amount}
    notifQueue.offer(job)
    worker = notifQueue.poll()
    AfricasTalking.sendSMS(worker.phone, worker.text)
```